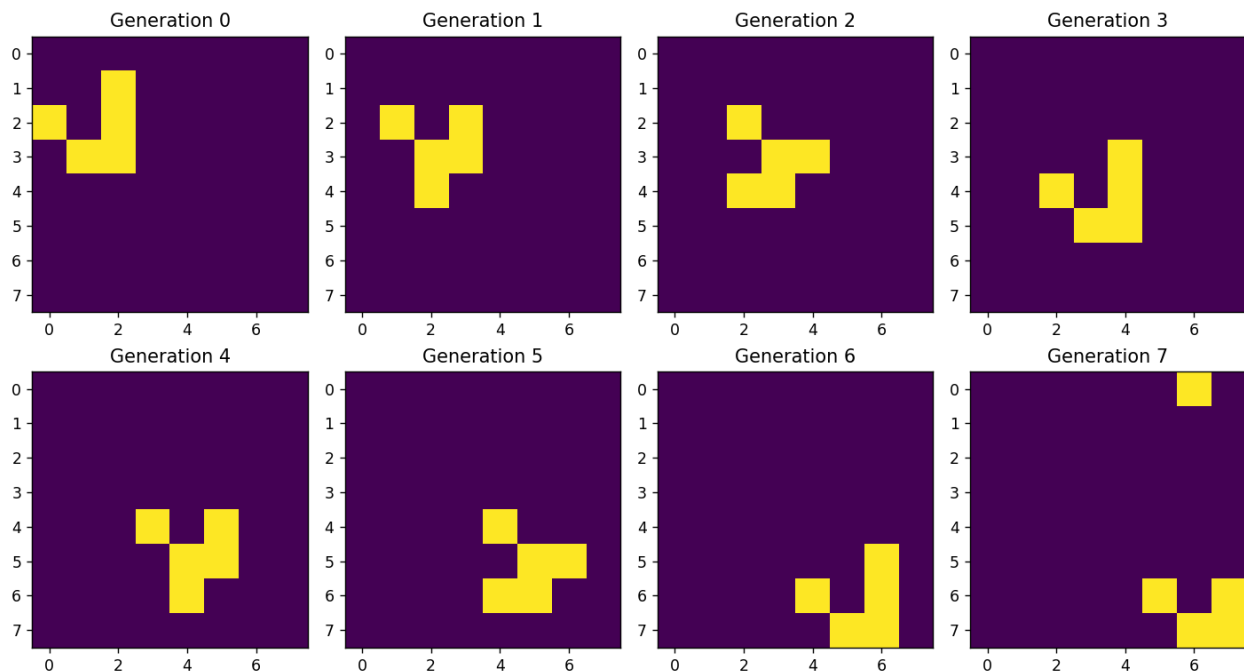


Computer Architecture Mini-Project III: Game of Life

Charlie Sands

10/26/2025

My selected variant of John Conway's Game of Life is a simple implementation of a single game that switches the color of all the active LEDs when it advances to the next generation. The colors cycle through solid red, green, blue then back to red indefinitely. The game has a cyclic boundary condition so any moving organism traversing the grid will loop around once it reaches the edge of the display. All succession rules of the game are implemented as per the original specification. The starting configuration of the game can be updated by modifying the binary data in the "pattern.txt" file. This file holds a 64-bit number with each eight-bit sequence representing the starting configuration of a row in the matrix. When my Verilog is synthesized in simulation mode (i.e. with Icarus Verilog through the testbench) lines of hardware description language are added to the synthesis by the preprocessor to export each generation of the game to a file called "memory_pattern.txt". I have also included preprocessor directives to decrease the length of delays in the game to allow for expedited analysis of the generational progression pattern of the simulation. I wrote a Python script which interprets the saved data and displays the board state for the first six generations and displays them with MATPLOTLIB. The graphics from this script are pictured below, showing the correct implementation of the Game of Life succession rules.



GitHub Repository for code [here](#).

Link to demonstration video [here](#).